

21世纪软件工程专业规划教材

软件工程导论（第6版）

第9章 编码、测试与维护

- 通常把编码和测试统称为实现。
- **编码**：就是把软件设计结果翻译成用某种程序设计语言书写的程序，是对设计的进一步具体化，程序的质量主要取决于软件设计的质量。
- **测试**：软件测试是保证软件质量的关键步骤（包括单元测试和综合测试），是对软件规格说明、设计和编码的最后复审。软件测试的工作量往往占软件开发总工作量的40%以上。
- **维护**：维护是软件生命周期的最后一个阶段，其基本任务是保证软件在一个相当长的时间里能够正常运行。

目录

CONTENTS

- 1 编码
- 2 软件测试基础
- 3 单元测试
- 4 集成测试
- 5 确认测试
- 6 白盒测试技术
- 7 黑盒测试技术
- 8 软件维护

Python	多范式（面向对象，函数式）	动态强类型	解释型（有JIT）	AI/ML，Web后端，自动化，数据分析
Java	面向对象	静态强类型	编译为字节码，JVM运行	企业应用，安卓开发，大型后端
C	过程式	静态弱类型	编译型	操作系统，嵌入式系统，硬件驱动
C++	多范式（面向对象，泛型）	静态弱类型	编译型	游戏，高性能软件，图形学，金融系统
JavaScript	多范式（原型面向对象）	动态弱类型	解释型/JIT编译	Web前端，全栈开发，跨平台App
Go	过程式，并发	静态强类型	编译型	云原生，微服务，网络工具，区块链
C#	面向对象，函数式	静态强类型	编译为字节码，.NET运行	Windows桌面，Unity游戏，企业后端
Rust	多范式（函数式影响）	静态强类型	编译型	系统编程，浏览器引擎，安全关键组件
Swift	面向对象，函数式	静态强类型	编译型	iOS/macOS原生应用开发
Kotlin	面向对象，函数式	静态强类型	编译为JVM字节码/JS/原生	安卓开发，Java后端替代
PHP	面向对象	动态弱类型	解释型	Web服务器端（尤其传统网站）
SQL	声明式	-	解释型	数据库查询与管理

软件测试基础

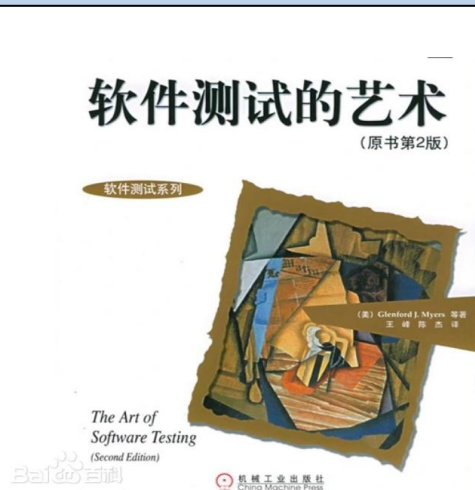
软件测试：从表面上看，软件测试的目的与软件工程所有其他阶段的目的都**相反**，其他阶段的目的都是“**建设性**”的，而测试阶段我们努力设计出一定的方案，其目的却是为了“**破坏**”已经建造好的软件系统——竭力证明程序中有错误，不能按照预定要求正确工作。

卧槽！
又有新bug！



暴露问题并不是软件测试的最终目的，发现问题是为了解决问题，目的是把一个高质量的软件产品交付给用户。

在G.J.Myers的经典著作《软件测试之艺术》（The Art of Software Testing）中，给出了测试的定义：“**程序测试是为了发现错误而执行程序的过程**”。



G.Myers给出的关于测试的一些规则如下：

- ① **测试**是为了发现程序中的错误而执行程序的过程。
- ② **好的测试**方案是极可能发现迄今为止尚未发现的错误的测试方案。
- ③ **成功的测试**是发现了至今为止尚未发现的错误的测试。

应该认识到测试决不能证明程序是正确的。即使经过了最严格的测试之后，仍然可能还有没被发现的错误潜藏在程序中。

软件测试准则

- ① **所有测试都应该能追溯到用户需求**——最严重的错误是导致程序不能满足用户需求的那些错误；
- ② **应该远在测试开始之前就制定出测试计划**——在编码之前就可以对所有测试工作进行计划 and 设计；
- ③ **早期测试**——测试活动应贯穿整个软件开发生命周期（SDLC）。需求评审、设计评审阶段就可以进行静态测试（如需求可测试性分析）。越早发现缺陷，修复成本越低。
- ④ **缺陷集群性**——缺陷往往不是均匀分布的，而是倾向于聚集在系统的某些复杂模块、新开发模块或频繁变更的模块中。帕累托法则：测试发现的错误中的80%很可能是由程序中20%的模块造成的；
- ⑤ **应该从“小规模”测试开始，并逐步进行“大规模”测试**——从小规模到大规模的测试递进，本质是“先验证正确性，再验证完整性”的工程思维；
- ⑥ **穷举测试是不可能的**——在测试中不可能执行每一个可能的路径；
- ⑦ **为了达到最佳的测试效果，应该由独立的第三方从事测试工作**——所谓“最佳效果”是指有最大可能性发现错误的测试。

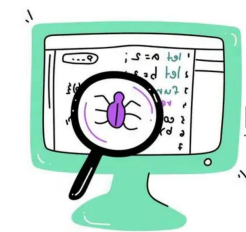
软件测试方法



软件测试步骤

除非是测试一个小程序，否则一开始就把整个系统作为一个单独的实体进行测试是不现实的。大型软件系统通常由若干个子系统组成，每个子系统又由许多模块组成，因此，大型软件系统的测试过程基本上由**模块测试**、**子系统测试**、**系统测试**、**验收测试**和**平行运行**等五个步骤组成。

① 模块测试——模块测试是最底层、最基础的测试



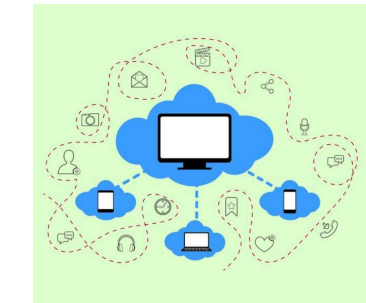
对于每个独立子功能的测试，通常容易设计检验模块正确性的测试方案。模块测试的目的是保证每个模块作为一个单元能正确运行，所以模块测试通常又称为单元测试。在这个测试步骤中所发现的往往是**编码**和**详细设计**的错误，一般由**编程人员**自己进行。

② 子系统测试——着重测试模块的接口

集成测试

子系统测试是把经过单元测试的模块放在一起形成一个子系统来测试。测试模块间的相互协作和通信是子系统测试的主要问题。

③ 系统测试——验证最终软件系统是否满足用户规定的需求



系统测试是把经过测试的子系统装配成一个完整的系统来测试。在这个过程中不仅应该发现**设计和编码**的错误，还应该验证系统确实能提供需求说明书中指定的功能，而且系统的动态特性也符合预定要求。在这个测试步骤中发现的往往是软件设计中的错误，也可能发现**需求说明中的错误**。

④ 验收测试——是部署软件之前的最后一个测试操作，也称为确认测试



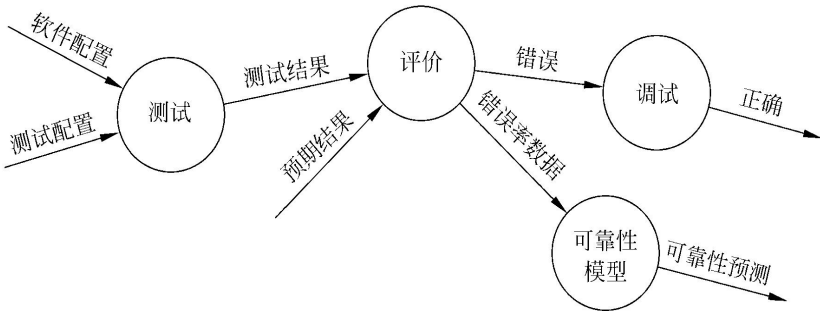
验收测试把软件系统作为单一的实体进行测试，测试内容与系统测试基本类似，但是它是在用**户积极参与下**进行的，而且可能主要使用**实际数据**（系统将来要处理的信息）进行测试。验收测试的目的是验证系统确实能够满足用户的需要，在这个测试步骤中发现的往往是**系统需求说明书中**的错误。

⑤ 平行运行——新旧系统的比较

所谓平行运行就是同时运行新开发出来的系统和将被它取代的旧系统，以便比较新旧两个系统的处理结果。这样做的具体目的有如下几点：

- (1) 可以在准生产环境中运行新系统而又不冒风险。
- (2) 用户能有一段熟悉新系统的时间。
- (3) 可以验证用户指南和使用手册之类的文档。
- (4) 能够以准生产模式对新系统进行全负荷测试，可以用测试结果验证性能指标。

测试阶段的信息流



左图描绘了测试阶段的信息流，这个阶段的输入信息有两类：
(1) 软件配置，包括需求说明书、设计说明书和源程序清单等；
(2) 测试配置，包括测试计划和测试方案。

单元测试

单元测试：单元测试集中检测软件设计的**最小单元**——

模块。它和编码属于软件过程的同一个阶段。在源程序代码通过编译程序的语法检查后，对**重要的执行通路**进行测试，以便发现模块内部的错误。

主要采用**白盒测试**技术，通过**人工测试**和**计算机测试**这样两种不同类型的测试方法，完成单元测试工作。

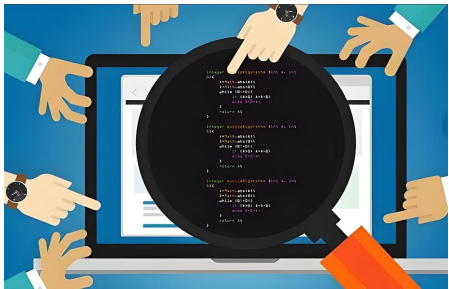


- ① **模块接口**——主要用于测试系统与外部其他系统之间的接口，以及系统内部各个子模块之间的接口。测试的重点是要检查接口参数传递的正确性，接口功能实现的正确性，输出结果的正确性，以及对各种异常情况的容错处理的完整性和合理性。
- ② **局部数据结构**——局部数据结构是为了保证临时存储在模块内的数据在程序执行过程中完整、正确的基础。模块的局部数据结构往往是错误的根源。
- ③ **重要的执行通路**——由于通常不可能进行穷尽测试，因此，在单元测试期间选择**最有代表性、最可能发现错误**的执行通路进行测试。
- ④ **出错处理通路**——好的设计应该能预见出现错误的条件，并且设置适当的处理错误的通路。不仅应该在程序中包含出错处理通路，而且应该认真测试这种通路。
- ⑤ **边界条件**——**边界测试**是单元测试中最后的也可能是最重要的任务。软件经常在边界上失效，边界条件测试是一项基础测试，也是后面系统测试中的功能测试的重点，边界测试执行的较好，可以大大提高程序健壮性。



代码审查

代码审查是指对计算机源代码系统化地审查，由审查小组正式对源程序进行**人工测试**。其目的是在找出及修正在软件开发初期未发现的错误，提升软件质量及开发者的技术。它是一种非常有效的程序验证技术，对于典型的程序来说，可以查出**30% ~ 70%**的**逻辑设计错误**和**编码错误**。



正式的代码审查：有审慎及仔细的流程，由多位参与者分阶段进行。由软件开发者参加一连串的会议，**一行一行的审查代码**，一般会使用打印好的原行码。正式的代码审查可以彻底的找到程序中的缺陷，但需要投入许多的资源。

结对编程：是两个程序员在一个计算机上共同工作，一个输入程序，另一个工程师审查他所输入的程序，结对编程是在极限编程中常见的开发方式。

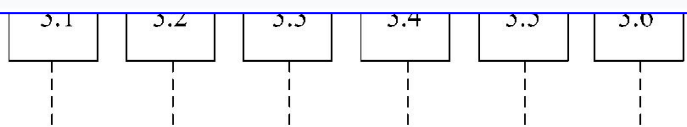
轻量型的非正式代码审查：需要投入的资源比正式的代码审查要少，一般会是在正常软件开发流程中同时进行，有时也会将结对编程视为轻量型代码审查的一种。



计算机测试

```
python

# 这是存根程序 (Stub)
def check_inventory_stub(product_id):
    # 它并不真的去数据库查询，只是硬编码返回一个值
    if product_id == "PHONE_001":
        return 100 # 假设库存有 100 件
    else:
        return 0
```



上图是一个**正文加工系统**的部分层次图，假定测试编号为3.0的关键模块——正文编辑模块。正文编辑模块不是一个独立的程序，需要有一个**测试驱动程序**来调用它。这个驱动程序说明必要的变量，接收测试数据——字符串，设置正文编辑模块的编辑功能。并且需要有**存根程序**简化地模拟正文编辑模块的**下层模块**来完成具体的编辑功能。

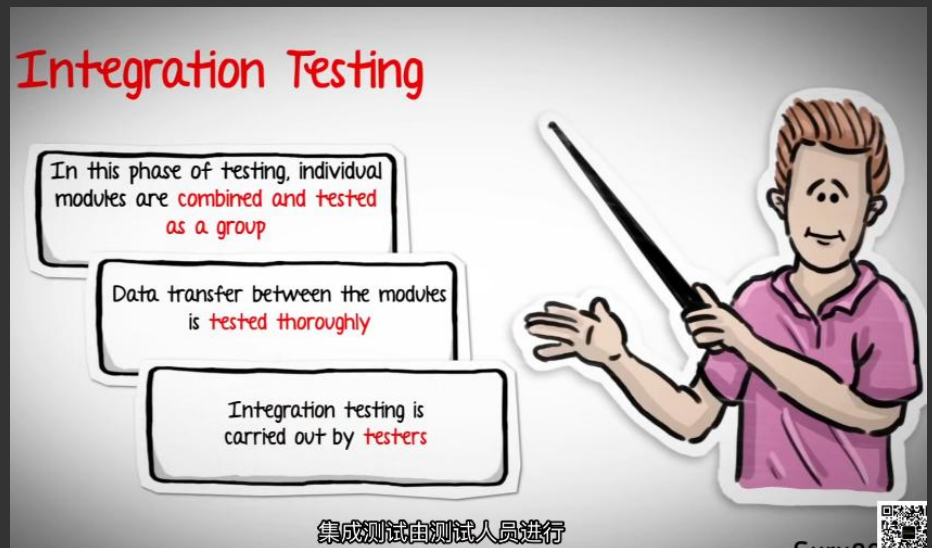
存根程序——存根程序代替被测试的模块所调用的模块（下层模块）。因此存根程序也可以称为“虚拟子程序”或“测试替身”。存根程序**最主要的作用**是将被测试代码与其依赖隔离开来。

集成测试

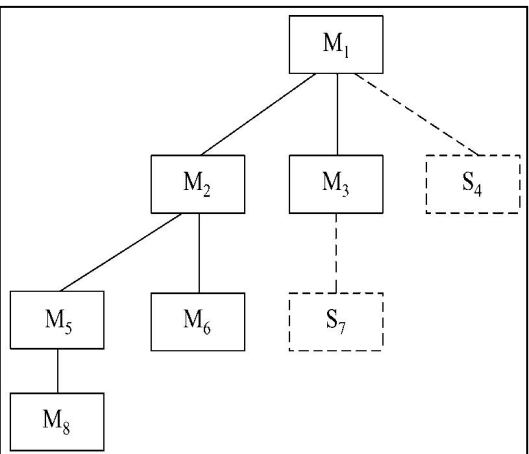
集成测试——也叫组装测试或联合测试。在单元测试的基础上，

将所有模块按照设计要求（如根据结构图）组装成为子系统或系统，进行集成测试（**非渐增式测试**和**渐增式测试**）。

实践表明，一些模块虽然能够单独地工作，但并不能保证连接起来也能正常的工作。一些局部反映不出来的问题，在全局上很可能暴露出来。它关注的是数据流、控制流和模块间的协作。



• **自顶向下集成方法**——是从**主控制模块**开始，沿着程序的**控制层次**向下移动，逐渐把各个模块结合起来。在把附属于（及最终附属于）主控制模块的那些模块组装到程序结构中去时，或者使用**深度优先**的策略，或者使用**宽度优先**的策略。



模块结合进软件结构的具体过程由下述4个步骤完成：

- ① **对主控制模块进行测试**，测试时用存根程序代替所有直接附属于主控制模块的模块；
- ② 根据**选定的结合策略**（深度优先或宽度优先），每次用一个实际模块代换一个存根程序（新结合进来的模块往往又需要新的存根程序）；
- ③ 在**结合进一个模块**的同时进行测试；
- ④ 为了保证加入模块没有引进新的错误，可能需要进行**回归测试**（即全部或部分地重复以前做过的测试）。

从②开始不断地重复进行上述过程，直到构造起完整的软件结构为止。

自顶向下的结合策略能够在测试的早期对**主要的控制**或**关键的抉择**进行检验。如果选择深度优先的结合方法，可以在早期实现软件的一个完整的功能并且验证这个功能。

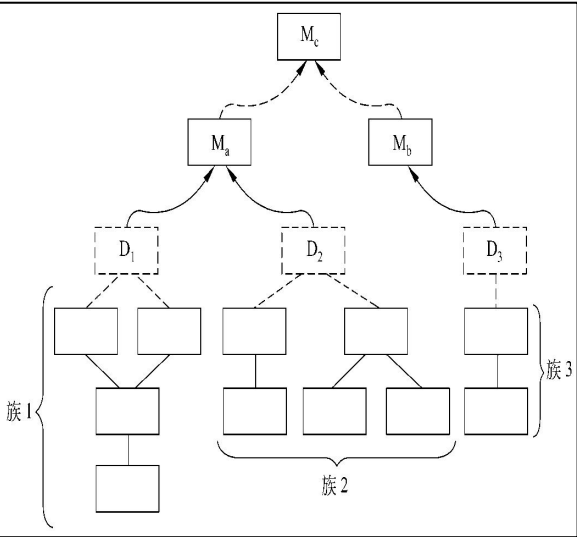
在自顶向下测试的初期，存根程序代替了低层次的模块，因此，在软件结构中没有重要的数据自下往上流。为了解决这个问题，测试人员有两种选择 ①把许多测试推迟到用真实模块代替了存根程序以后再进行；②从层次系统的底部向上组装软件。

深度优先的结合方法先组装在软件结构的一条主控制通路上的所有模块。选择一条主控制通路取决于应用的特点，并且有很大任意性。

宽度优先的结合方法是沿软件结构水平地移动，把处于同一个控制层次上的所有模块组装起来。

自底向上集成

自底向上测试从“原子”模块（即在软件结构最低层的模块）开始组装和测试。因
为是从底部向上结合模块，总能得到所需的下层模块处理功能，**所以不需要存根程序**。



- 用下述步骤可以实现自底向上的结合策略。
- ① 把低层模块组合成实现某个特定软件子功能的族；
 - ② 写一个驱动程序（用于测试的控制程序），协调测试数据的输入和输出；
 - ③ 对由模块组成的子功能族进行测试；
 - ④ 去掉驱动程序，沿软件结构自下向上移动，把子功能族组合起来形成更大的子功能族。
- 上述第②～④步实质上构成了一个循环。

维度	自底向上	自顶向下
起点	底层模块	顶层模块
存根需求	需要测试驱动	需要存根
早期验证	基础设施	用户界面/主流程
发现问题时机	早期发现底层问题	早期发现架构问题
适合项目	技术驱动、底层复杂	客户驱动、UI重要
并行开发	更容易	较困难

回归测试

回归测试——是指在修改了旧代码后（如修复缺陷、增加新功能、优化性能），重新
执行之前已经通过的测试用例，以确保这些修改没有引入新的错误或导致现有功能出现
倒退（即“回归”）。



在集成测试过程中，每当一个新模块结合进来时，
程序就发生了变化：建立了新的数据流路径，
可能出现了新的I/O操作，激活了新的控制逻辑。
在集成测试的范畴中，回归测试是指重新执行
已经做过的测试的某个子集，以保证上述这些变
化没有带来非预期的副作用。

回归测试的类型：

- ① **修正性回归测试**：修复一个具体缺陷后，验证这个缺陷是否被正确修复；
- ② **增量性回归测试**：测试新功能本身，并测试其与现有系统的集成部分；
- ③ **完全/全面回归测试**：当代码有重大修改时，执行完整的测试套件。
- ④ **选择性/部分回归测试**：只执行受代码变更影响的相关测试用例。

回归测试的核心是“防退化”

方面	确认测试	验证测试
核心问题	“我们是否在构建正确的东西？”	“我们是否在正确地构建东西？”
关注点	外部行为，用户需求、业务目标	内部结构，代码质量、设计规范、无缺陷
执行级别	高级别（系统级、验收级）	低级别（单元级、集成级）
评判依据	需求规格说明书、用户期望	设计文档、编码规范、测试用例
执行者	通常由 测试团队 和 最终用户/客户 （验收时）执行	通常由 开发人员 （单元测试）和 测试工程师 （集成测试）执行
类比	制造一辆汽车 ：检查它是否符合客户订单上要求的型号、颜色、功能。	制造一辆汽车 ：检查每个零件（发动机、轮胎）是否按照工程图纸正确制造和组装。



<https://baike.baidu.com/item/%E9%AA%8C%E6%94%B6%E6%B5%8B%E8%AF%95/10914477>

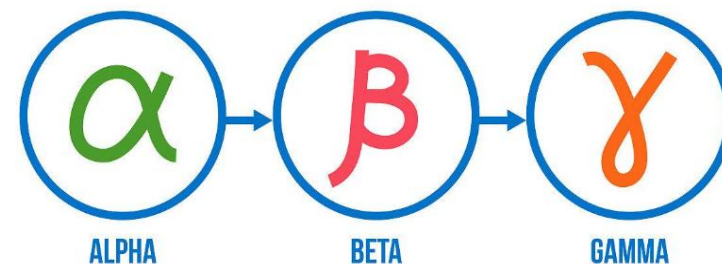
确认测试通常使用**黑盒测试法**。应该仔细设计测试计划和测试过程，测试计划包括要进行的测试的种类及进度安排，测试过程规定了用来检测软件是否与需求一致的测试方案。

确认测试有下述两种可能的结果：**(1)** 功能和性能与用户要求一致，软件是可以接受的。**(2)** 功能和性能与用户要求有差距。

Alpha和Beta测试

如果一个软件是为**许多客户**开发的（例如，向大众公开出售的盒装软件产品），那么绝大多数软件开发商都使用被称为**Alpha测试**和**Beta测试**的过程，来发现那些看起来只有最终用户才能发现的错误。

SOFTWARE TESTING PHASES



Alpha测试：由用户在**开发者的场所**进行，并且在开发者对用户的“**指导**”下进行测试。开发者负责记录发现的错误和使用中遇到的问题。Alpha测试是在**受控**的环境中进行的。

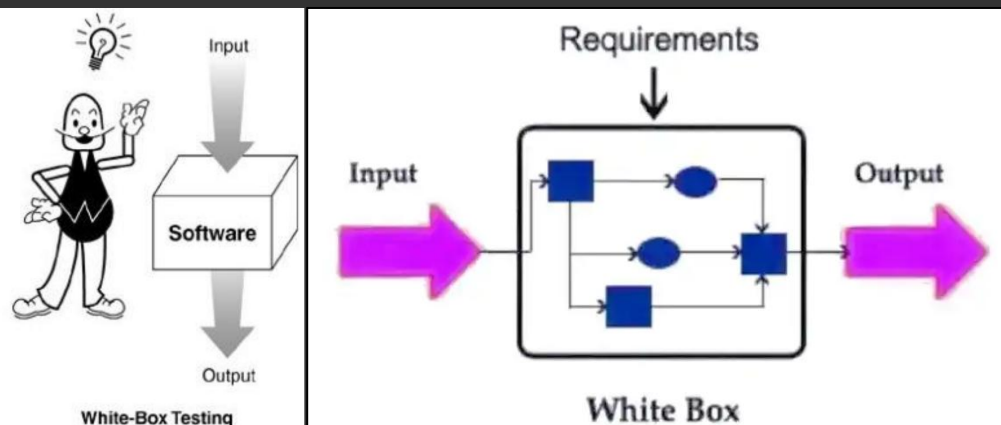
Beta测试：由软件的最终用户们在一个或多个**客户场所**进行。与Alpha测试不同，开发者通常不在Beta测试的现场。Beta测试是软件在开发者**不能控制**的环境中的“真实”应用。（游戏内测/公测）

白盒测试技术

测试方案：是在项目启动或一个新版本/特性开始测试前，制定的综合性、纲领性测试计划文档。它定义了测试活动的“框架”和“蓝图”。包括具体的**测试目的、测试方法、测试时间安排、应输入的测试数据和预期结果等。**

测试用例：通常把测试数据和预期的输出结果称测试用例。

设计测试方案是测试阶段的关键技术问题。

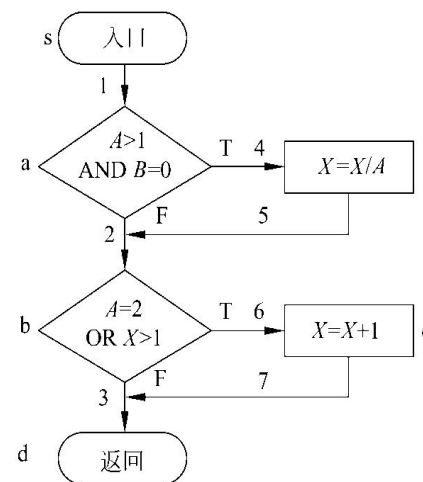
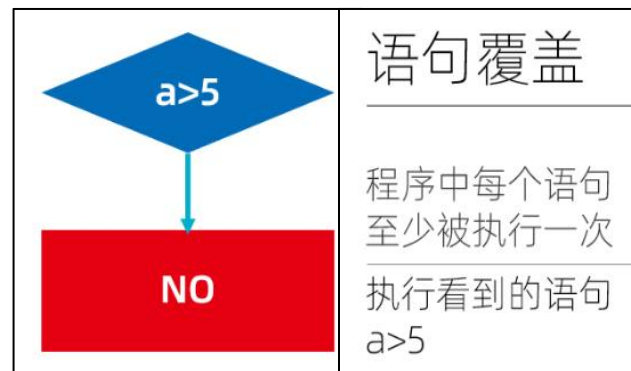


白盒测试——（又称结构测试），是一种**测试用例设计方法**，是把程序看成装在一个透明的白盒子里，测试者**完全知道程序的结构和处理算法**。**核心是基于源代码的内部逻辑进行测试**，检测程序中的主要执行通路是否都能按预定要求正确工作。

逻辑覆盖

有选择地执行程序中某些**最有代表性的通路**是对穷尽测试的唯一可行的替代办法。**逻辑覆盖**是对一系列测试过程的总称，这组测试过程逐渐进行越来越完整的通路测试。

① **语句覆盖**——选择足够多的测试数据，使被测程序中**每个语句至少执行一次**。



左图为被测试模块的流程图。为了使每个语句都执行一次，程序的执行路径应该是**sacbed**,为此只需要输入下面的测试数据（实际上**X**可以是任意实数）：**A=2, B=0, X=4**。

语句覆盖对程序的逻辑覆盖很少，在上面例子中两个判定条件都只测试了条件为真的情况，如果条件为假时处理有错误，显然不能发现。

语句覆盖只关心判定表达式的值，而没有分别测试判定表达式中每个条件取不同值时的情况。为了执行**sacbed**路径，以测试每个语句，只需两个判定表达式 **$(A > 1) \text{ AND } (B = 0)$** 和 **$(A = 2) \text{ OR } (X > 1)$** 都取真值，因此使用上述一组测试数据就够了。但是，如果程序中把第一个判定表达式中的逻辑运算符**AND**错写成**OR**，或把第二个判定表达式中的条件 **$X > 1$** 误写成 **$X < 1$** ，使用上面的测试数据并不能查出这些错误。

综上所述，可以看出**语句覆盖是很弱的逻辑覆盖标准**。

- ② **判定覆盖**——又叫分支覆盖，它的含义是，不仅每个语句必须至少执行一次，而且每个判定的每种可能的结果都应该至少执行一次，**每个判断（分支）的真（True）和假（False）两种结果至少各出现一次。**

判定覆盖

程序中的每个分支至少都通过一次

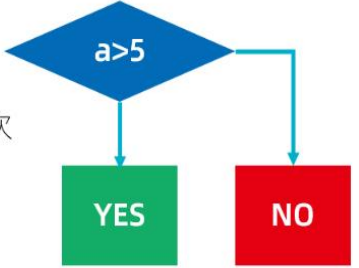
每个判定真假各一次

$a>5$ $a\leq 5$

如果还有 $c>0$ 只需

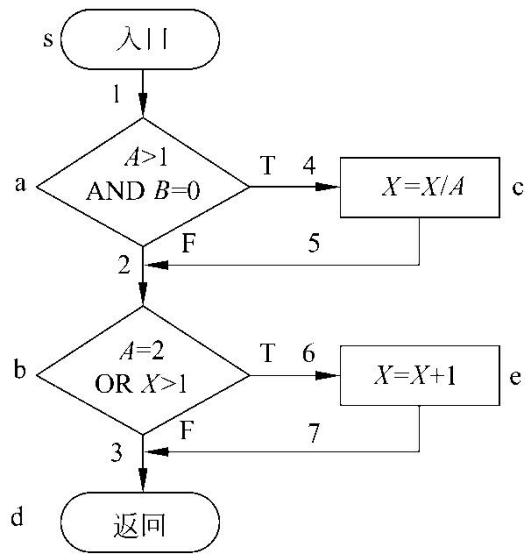
$a>5,c>0$

$a\leq 5,c\leq 0$



判定覆盖比语句覆盖强，但是对程序逻辑的覆盖程度仍然不高，

例如，下面的测试数据只覆盖了程序全部路径的一半。



对于上述例子来说，能够分别覆盖路径 **sacbed**和**sabd**的两组测试数据，或者可以分别覆盖路径**sacbd**和**sabed**的两组测试数据，都满足判定覆盖标准。例如，以下两组测试数据就可做到判定覆盖：

- ① **A=3, B=0, X=3** （覆盖**sacbd**）
- ② **A=2, B=1, X=1** （覆盖**sabed**）

- ③ **条件覆盖**——不仅每个语句至少执行一次，而且设计足够的测试用例，使得程序中**每个判断内的每个条件的可能取值（真/假）至少各出现一次。**

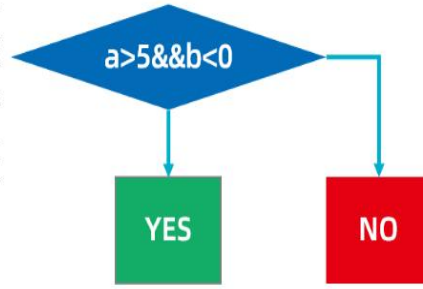
条件覆盖

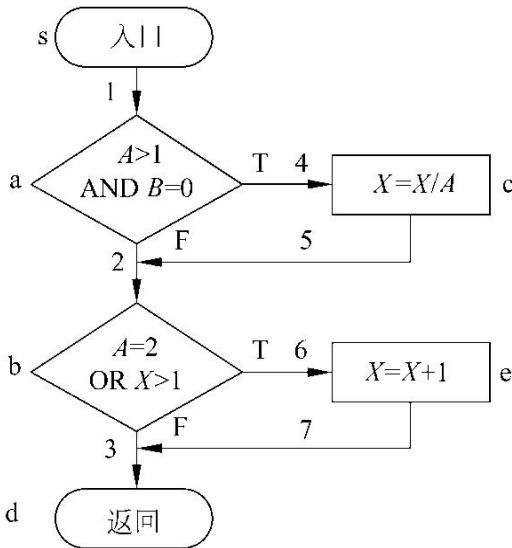
判定中每个条件获得各种可能的结果

每种条件取值一次

$a>5 \ \&\& \ b>0$ no

$a\leq 5 \ \&\& \ b<0$ no





上例中共有两个判定表达式，每个表达式中有两个条件，为了做到条件覆盖，应该选取测试数据满足下面的要求。在**a**点有下述各种结果出现：**A>1,A≤1,B=0,B≠0**；在**b**点有下述各种结果出现：**A=2,A≠2,X>1,X≤1**；

- 只需要使用下面两组测试数据就可以达到上述覆盖标准：

 - ① $A=2,B=0,X=4$ （满足 $A>1,B=0,A=2$ 和 $X>1$ ，执行路径**sacbed**）
 - ② $A=1,B=1,X=1$ （满足 $A\leq 1,B\neq 0,A\neq 2$ 和 $X\leq 1$ ，执行路径**sabd**）

条件覆盖通常比判定覆盖**强**，但**满足条件覆盖的测试数据不一定满足判定覆盖**。如果使用下面两组测试数据，则只满足条件覆盖标准并不满足判定覆盖标准（第二个判定表达式的值总为真）：

 - ① $A=2,B=0,X=1$ （满足 $A>1,B=0,A=2$ 和 $X\leq 1$ ，执行路径**sacbed**）
 - ② $A=1,B=1,X=2$ （满足 $A\leq 1,B\neq 0,A\neq 2$ 和 $X>1$ ，执行路径**sabed**）

④ **判定/条件覆盖**——是一种能**同时满足判定覆盖和条件覆盖的逻辑覆盖**，它的含义是，选取足够多的测试数据，使得判定表达式中的每个条件都取到各种可能的值，而且每个判定表达式也都取到各种可能的结果。

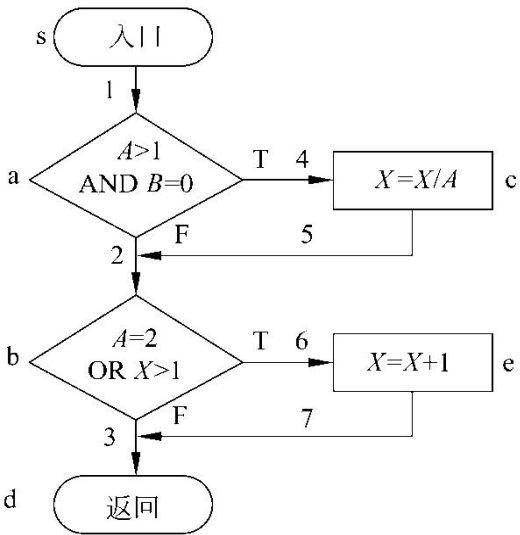
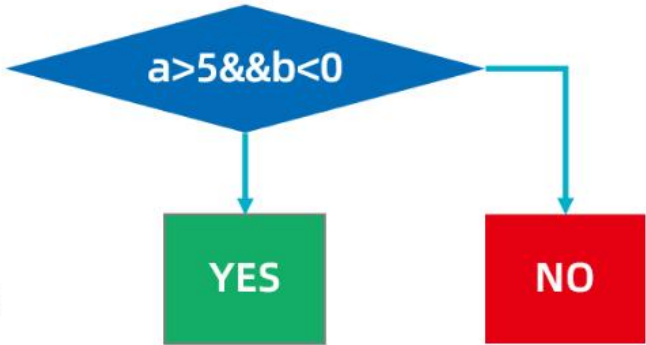
判断-条件覆盖

判定中的每个条件取到各种可能的值和结果

每种条件取值一次

$a > 5 \ \&\& \ b < 0$ yes

$a \leq 5 \ \&\& \ b \geq 0$ no



对于上例而言，下述两组测试数据满足判定/条件覆盖标准：

① **A=2,B=0,X=4**

② **A=1,B=1,X=1**

但是，这两组测试数据也就是为了满足条件覆盖标准最初选取的两组数据，因此，**有时判定/条件覆盖也并不比条件覆盖更强。**

⑤ **条件组合覆盖**——是更强的逻辑覆盖标准，它要求选取足够多的测试数据，**使得每个判定表达式中条件的各种可能组合都至少出现一次。**

条件组合覆盖

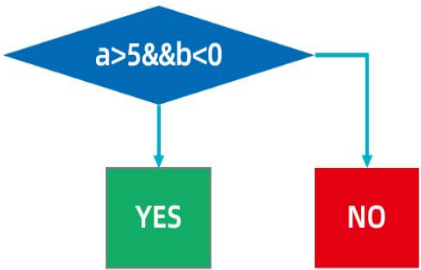
每个判定中条件的各种可能组合都至少出现一次

$a > 5 \ \&\& \ b < 0$

$a > 5 \ \&\& \ b \geq 0$

$a \leq 5 \ \&\& \ b < 0$

$a \leq 5 \ \&\& \ b \geq 0$



- (1) $A > 1, B = 0$
- (2) $A > 1, B \neq 0$
- (3) $A \leq 1, B = 0$
- (4) $A \leq 1, B \neq 0$
- (5) $A = 2, X > 1$
- (6) $A = 2, X \leq 1$
- (7) $A \neq 2, X > 1$
- (8) $A \neq 2, X \leq 1$

下面的4组测试数据使上面列出的8种条件组合每种至少出现一次：

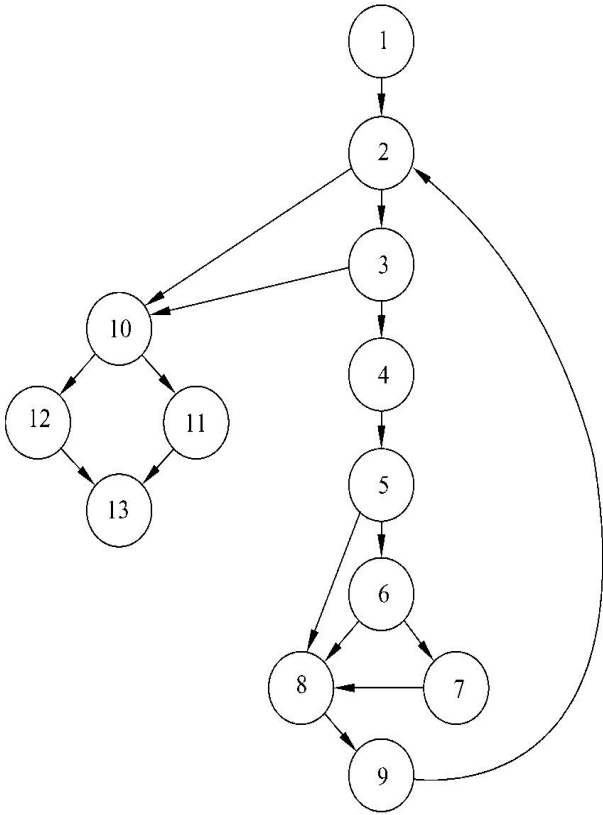
- ① **A=2,B=0,X=4**（针对(1)和(5)，执行路径sacbed）
- ② **A=2,B=1,X=1**（针对(2)和(6)，执行路径sabcd）
- ③ **A=1,B=0,X=2**（针对(3)和(7)执行路径sabcd）
- ④ **A=1,B=1,X=1**（针对(4)和(8)，执行路径sabd）

显然，满足条件组合覆盖标准的测试数据，也一定满足判定覆盖、条件覆盖和判定/条件覆盖标准。因此，**条件组合覆盖是前述几种覆盖标准中最强的。**但是，满足条件组合覆盖标准的测试数据并不一定能使程序中的每条路径都执行到，例如，上述4组测试数据都没有测试到路径sacbd。

① **基本路径测试**——一种白盒测试技术。使用基本路径测试设计测试用例时，首先计算程序的**环形复杂度**，并用该复杂度为指南定义**执行路径的基本集合**，从该基本集合导出的测试用例可以保证程序中的每条语句至少执行一次，而且每个条件在执行时都将分别取真、假两种值。

A. 根据过程设计结果画出相应的流程图

```
1: i=1;
   total.input=total.valid=0;
   sum=0;
2: DO WHILE value[i] <> -999
3:   AND total.input<100
4:   increment total.input by1;
5:   IF value[i]>=minimum
6:     AND value[i]<=maximum
7:   THEN increment total.valid by 1;
   sum=sum+value[i];
8:   ENDIF
   increment i by 1;
9: ENDDO
10: IF total.valid>0
11: THEN average=sum/total.valid;
12: ELSE average=-999;
13: ENDIF
   END average
```



B. 计算流图的环形复杂度

环形复杂度定量度量程序的逻辑复杂性。使用第6.5.1小节讲述的3种方法之一计算环形复杂度。经计算，流图的环形复杂度为**6**。

C. 确定线性独立路径的基本集合

独立路径是指至少引入程序的一个新处理语句集合或一个新条件的路径，即独立路径**至少包含一条在定义该路径之前不曾用过的边**。

程序的环形复杂度决定了程序中独立路径的数量，而且这个数是确保程序中所有语句至少被执行一次所需的测试数量的上界。

上述程序的环形复杂度为**6**，因此共有**6**条独立路径。

- 路径1: 1-2-10-11-13
- 路径2: 1-2-10-12-13
- 路径3: 1-2-3-10-11-13
- 路径4: 1-2-3-4-5-8-9-2-...
- 路径5: 1-2-3-4-5-6-8-9-2-...
- 路径6: 1-2-3-4-5-6-7-8-9-2-...

D. 设计可强制执行基本集合中每条路径的测试用例。

应该选取测试数据使得在测试每条路径时都适当地设置好各个判定结点的条件。测试第③步得出的基本集合的测试用例如下：

路径1的测试用例：

value[k]=有效输入值，其中 $k < i$ （i的定义在下面）

value[i]=-999,其中 $2 \leq i \leq 100$

预期结果：基于k的正确平均值和总数

注意，路径1无法独立测试，必须作为路径4或5或6的一部分来测试。

路径2的测试用例：

value[1]=-999

预期结果： average=-999,其他都保持初始值

路径3的测试用例：

试图处理101个或更多个值，前100个数值应该是有效输入值

预期结果：前100个数的平均值，总数为100

注意，路径3无法独立测试，必须作为路径4或5或6的一部分来测试。

路径4的测试用例：

value[i]=有效输入值，其中 $i < 100$

value[k]<minimum，其中 $k < i$

预期结果：基于k的正确平均值和总数

路径5的测试用例：

value[i]=有效输入值，其中 $i < 100$

value[k]>maximum，其中 $k < i$

预期结果：基于k的正确平均值和总数

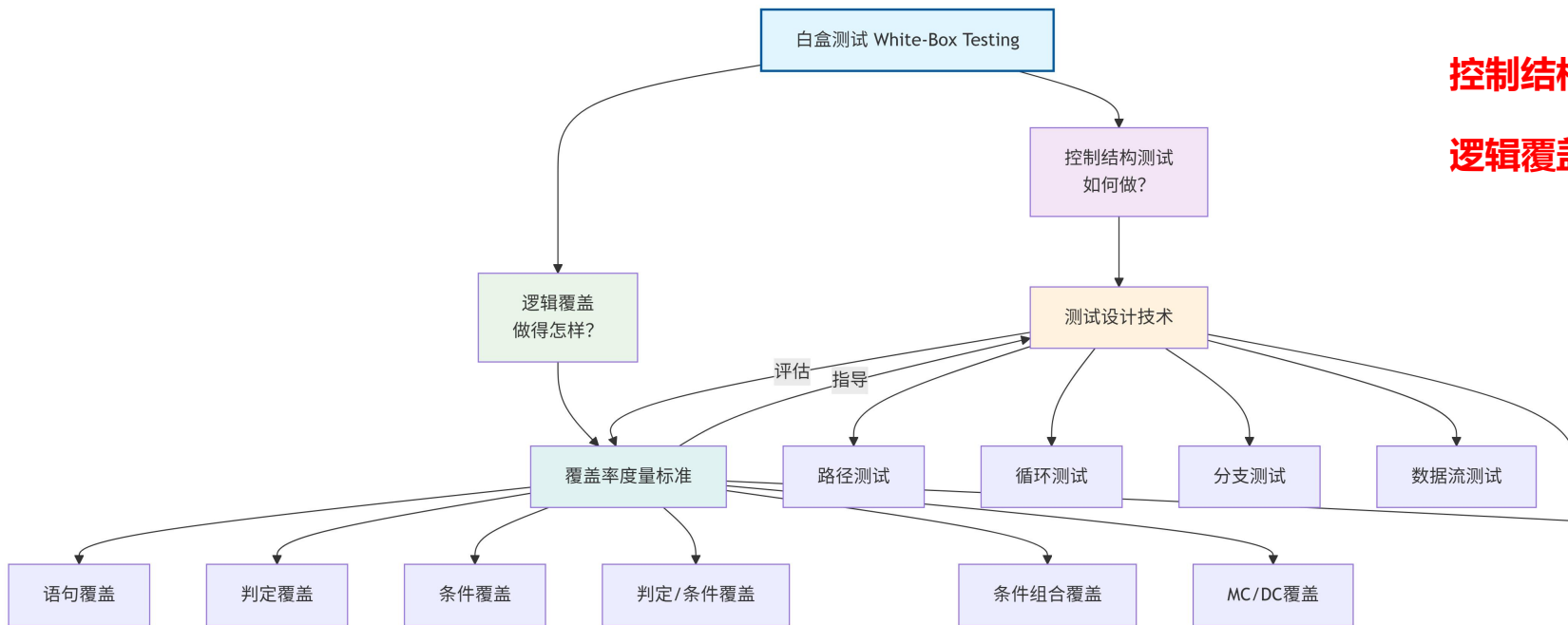
路径6的测试用例：

value[i]=有效输入值，其中 $i < 100$

预期结果：正确的平均值和总数

在测试过程中，执行每个测试用例并把实际输出结果与预期结果相比较。一旦执行完所有测试用例，就可以确保程序中所有语句都至少被执行了一次，而且每个条件都分别取过true值和false值。

注意：某些独立路径（例如，本例中的路径1和路径3）不能以独立的方式测试，例如，为了执行本例中的路径1，需要满足条件total.valid>0。在这种情况下，这些路径必须作为另一个路径的一部分来测试。



控制结构测试指导我们如何设计测试用例

逻辑覆盖告诉我们这些用例的充分性如何

两者的关联与依赖关系

1. 控制结构测试是实现逻辑覆盖的主要手段

要想达到高逻辑覆盖率，必须运用各种控制结构测试技术。

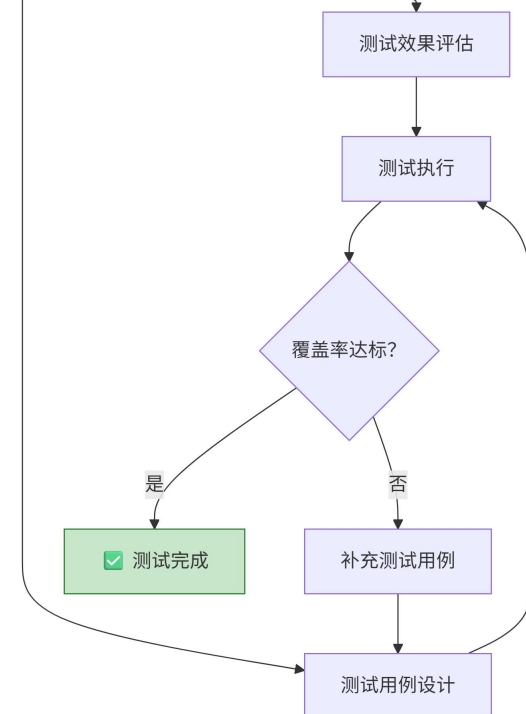
例如，为了实现“判定覆盖”，你必须进行“分支测试”。

2. 逻辑覆盖是评估控制结构测试效果的核心指标

使用的测试技术好不好看看覆盖率就知道了，低覆盖率意味着当前的控制结构测试策略需要加强。

3. 在测试过程中形成一个闭环：

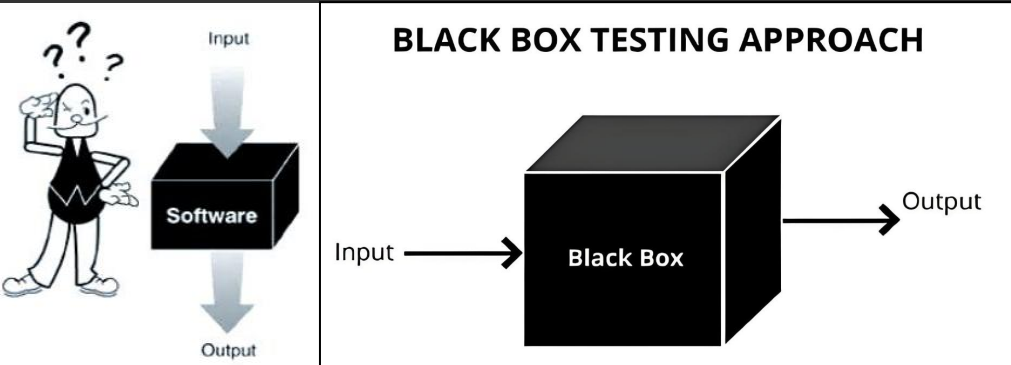
分析代码控制结构 → 应用控制结构测试技术设计用例
→ 执行测试 → 用逻辑覆盖标准评估 → 发现覆盖率不足 → 补充设计用例（再次应用控制结构测试） → 达成覆盖率目标



黑盒测试技术

黑盒测试——它是通过测试来检测**每个功能是否都能正常使用**。

在测试中，把程序看作一个不能打开的黑盒子，在完全不考虑程序内部结构和内部特性的情况下，在程序接口进行测试，它只检查程序功能**是否按照需求规格说明书的规定正常使用**，程序是否能适当地接收输入数据而产生正确的输出信息。黑盒测试着眼于程序**外部结构**，不考虑内部逻辑结构，主要针对**软件界面和软件功能**进行测试。



白盒测试在测试过程的**早期**阶段进行，而黑盒测试主要用于测试过程的**后期**。力图发现下述类型的错误：功能不正确或遗漏了功能；界面错误；数据结构错误或外部数据库访问错误；性能错误初始化和终止错误。

等价划分

该方法将程序所有可能的输入或输出数据划分为**有效等价类**和**无效等价类**，通过选取每类中的**代表性数据**设计测试用例进行测试，以保证测试的完整性和代表性。

划分等价类需要经验，下述的**启发式规则**可能有助于等价类划分。

- (1) 如果规定了输入值的范围，则可划分出一个有效的等价类（输入值在此范围内），两个无效的等价类（输入值小于最小值或大于最大值）
- (2) 如果规定了输入数据的个数，则类似地也可以划分出一个有效的等价类和两个无效的等价类。
- (3) 如果规定了输入数据的一组值，而且程序对不同输入值做不同处理，则每个允许的输入值是一个有效的等价类，此外还有一个无效的等价类（任一个不允许的输入值）。

步骤一：划分等价类

输入及外部条件	有效等价类	无效等价类
报表日期的类型及长度	6 位数字字符①	有非数字字符④ 少于 6 个数字字符⑤ 多于 6 个数字字符⑥
年份范围	在 2001 ~ 2005 之间②	小于 2001 ⑦ 大于 2005 ⑧
月份范围	在 01 ~ 12 之间③	小于 01 ⑨ 大于 12 ⑩

假设有一个把数字串转变成整数的函数。运行程序的计算机字长**16位**，用二进制补码表示整数。这个函数是用**Pascal**语言编写的，它的说明如下：

```
function strtoint (dstr:shortstr):integer;
```

函数的参数类型是**shortstr**，它的说明是：

```
type shortstr=array[1..6] of char;
```

被处理的数字串是右对齐的，也就是说，如果数字串比**6**个字符短，则在它的左边补空格。如果数字串是负的，则负号和最高位数字紧相邻（负号在最高位数字左边一位）。

考虑到**Pascal**编译程序固有的检错功能，测试时不需要使用长度不等于**6**的数组做实际参数，更不需要使用任何非字符数组类型的实在参数。

编号	描述	输入	预期输出
6	太大的正整数	'132767'	错误——无效输入
7	空字符串	' '	错误——没有数字
8	字符串左部字符既不是零也不是空格	'xxxxx1'	错误——填充错
9	最高位数字后面有空格	' 1 2'	错误——无效输入
10	最高位数字后面有其他字符	' 1x2'	错误——无效输入
11	负号和最高位数字之间有空格	' - 12'	错误——负号位置错

经验表明，**处理边界情况时程序最容易发生错误**。该方法通过选取输入/输出范围的边界值作为测试数据，重点检测数据边界附近的异常行为，例如，许多程序错误出现在下标、纯量、数据结构和循环等等的边界附近。因此，设计使程序运行在边界情况附近的测试方案，暴露出程序错误的可能性更大一些。

使用**边界值分析方法**设计测试方案首先应该确定边界情况，通常输入等价类和输出等价类的边界。选取的测试数据应该刚好等于、刚刚小于和刚刚大于边界值。

为了测试前述的把数字串转变成整数的程序，除了上一小节已经用等价划分法设计出的测试方案外，还应该用边界值分析法再补充下述测试方案。

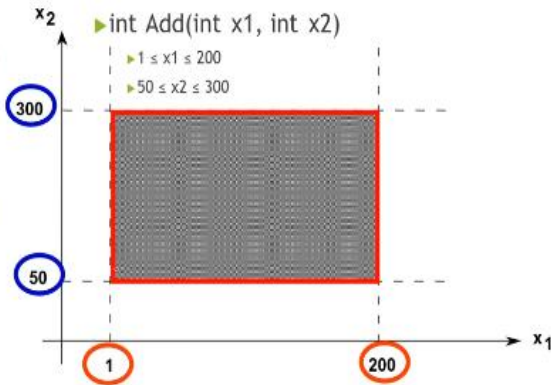
编号	描述	输入	预期输出
1	使输出刚好等于最小的负整数	'-32768'	-32768
2	使输出刚好等于最大的正整数	'32767'	32767
3	使输出刚刚小于最小的负整数	'-32769'	错误——无效输入
4	使输出刚刚大于最大的正整数	'32768'	错误——无效输入

1.1 边界在哪里

该加法函数有两个输入条件，x1和x2，取值范围如下图

显然x1条件在有效域内具有最小值1和最大值200，那么1和200就是x1的两个边界点；50和300就是x2的两个边界点。

对于这个加法函数来说边界就是由这4个边界点构成的、红色矩形上的所有点的集合。也就是说寻找被测系统的边界实际上就是寻找每个输入条件的边界点。



为了说明这4个问题，我们假设了如下的一个测试：

这是一个包含2个输入值的加法函数，x1和x2的取值范围都是闭区间，这个案例该如何设计边界值测试用例呢！

同学们大多数可能会设计成如下的测试用例：

编号	x1	x2	预期输出
1	1	50	51
2	200	300	500
3	1	300	301
4	200	50	250
5	0	49	-1
6	201	301	-1
7	2	51	53
8	199	299	498

那么，为什么呢！我们——来破解。

int Add(int x1, int x2)

- $1 \leq x1 \leq 200$
- $50 \leq x2 \leq 300$
- 需求简述：
 - 对于有效输入，函数返回 x1 与 x2 的和；
 - 对于无效输入，函数返回 -1；

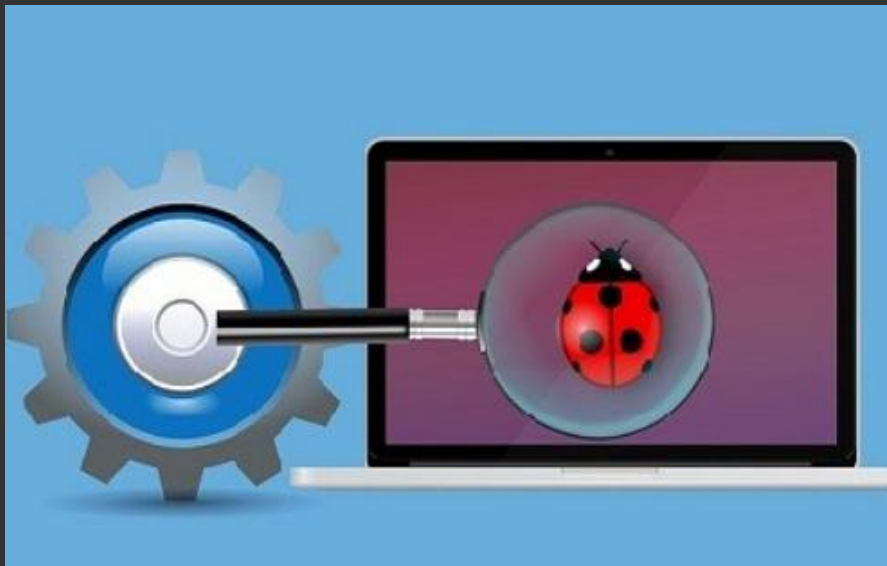
其实不然，我们认为下面这个才是较好的测试用例：

编号	x1	x2	预期输出
1	0	200	-1
2	1	200	201
3	2	200	202
4	199	200	399
5	200	200	400
6	201	200	-1
7	100	49	-1
8	100	50	150
9	100	51	151
10	100	299	399
11	100	300	400
12	100	301	-1

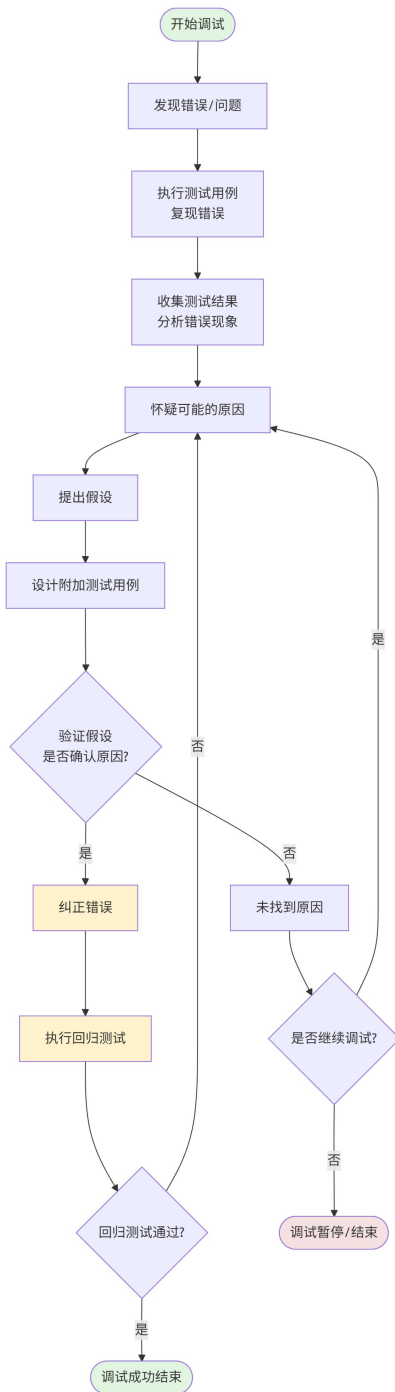


软件调试

软件调试——是软件开发、程序运行过程中，定位、分析并修复程序错误 / 异常的核心过程，也是编程和软件工程中不可或缺的环节。预期、稳定可靠地运行。



调试的核心目标不是“**重写代码**”，而是精准**定位问题**根源并“**最小化修复问题**”，同时**避免引入新的错误**，最终保证程序的功能、逻辑、性能都符合要求。



调试要解决的程序问题主要分3类：

- ① **语法错误**：最基础的错误（如少写分号、变量名拼写错误、括号不匹配），多数编译器 / 解释器会直接报错，定位难度低，属于入门级调试内容；
- ② **运行时错误**：程序能启动，但运行中崩溃 / 抛出异常（如数组越界、空指针引用、文件找不到、数据类型不匹配），需要跟踪程序执行过程才能定位；
- ③ **逻辑错误**：程序不崩溃、无报错，但结果不符合预期（如计算结果错误、流程分支走偏、循环次数异常），这是调试中最复杂的类型，需要验证程序的执行逻辑和数据流转。

调试的方法从简单到复杂分为：

- ① **打印调试**：最基础的方法，在代码关键位置加打印语句（如 Python 的 `print()`），输出变量值、执行流程，验证数据是否符合预期；
- ② **日志调试**：通过日志框架（如 Python 的 `logging`）输出不同级别的日志，适合线上程序、长流程程序的调试；
- ③ **断点调试**：最常用的高效方法，通过开发工具（PyCharm/IDEA）在代码指定行设置断点，运行程序时会在断点处暂停，可逐行执行代码、查看变量实时值、跟踪调用栈，精准定位逻辑错误；
- ④ **单元测试调试**：为单个函数 / 模块编写测试用例，通过运行测试用例，快速定位哪个模块 / 函数出问题，适合大型项目的模块化调试；
- ⑤ **注释调试**：临时注释掉疑似有问题的代码段，逐步还原，排查问题代码的范围，适合新手快速定位简单问题。

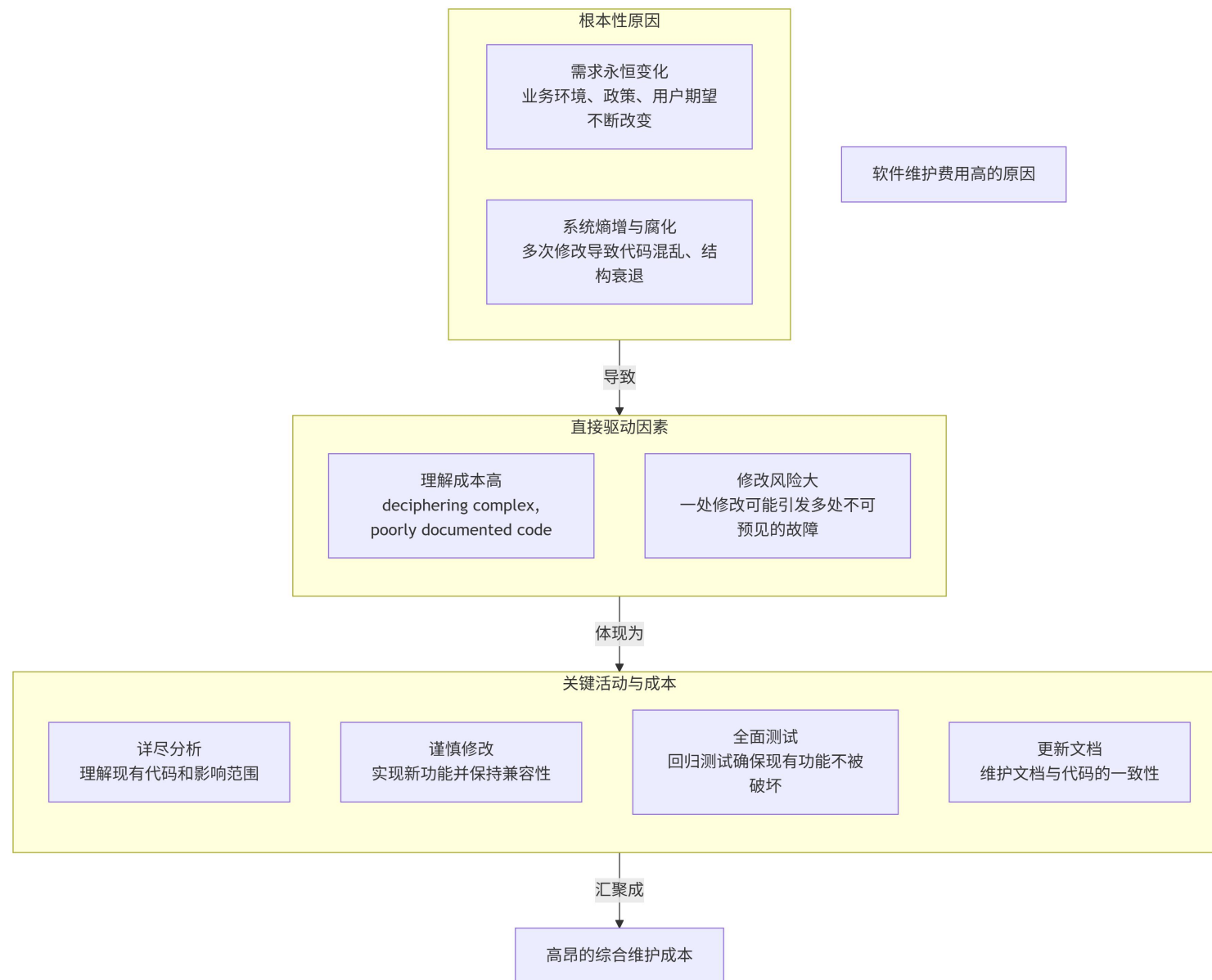
软件维护

软件维护——是一个软件工程名词，是指在软件产品发布后，因修正错误、提升性能或其他属性而进行的软件修改。这个阶段是软件生命周期的最后一个阶段，其基本任务是保证软件在一个相当长的时期能够正常运行。



软件维护需要的工作量很大，平均说来，大型软件的维护成本高达开发成本的**4倍**左右。**软件工程的主要目的就是要提高软件的可维护性，减少软件维护所需要的工作量，降低软件系统的总成本。**

软件维护的原因



软件维护的定义

所谓软件维护就是在软件已经交付使用之后，为了改正错误或满足新的需要而修改软件的过程。可以通过描述软件交付使用后可能进行的4项活动，具体地定义软件维护。



改正性维护：因为软件测试不可能暴露出一个大型软件系统中所有潜藏的错误，使用期间用户必然会发现程序错误，并且把他们遇到的问题报告给维护人员。把**诊断和改正错误**的过程称为改正性维护。



适应性维护：当操作系统升级、硬件设备更新或数据库迁移时，需通过调整软件代码或架构来维持功能。其占比约**25%**且与用户需求直接相关。该维护类型强调**技术文档同步更新**和**可移植性设计**，提升系统对环境变迁的适应能力。

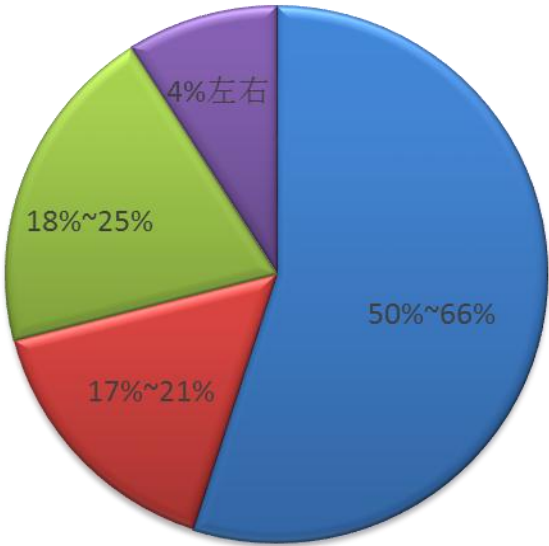


完善性维护：当一个软件系统顺利地运行时，在使用软件的过程中用户往往提出**增加新功能或修改已有功能**的建议，还可能提出一般性的改进意见。为了满足这类要求，需要进行完善性维护。这项维护活动通常占软件维护工作的大部分。



预防性维护：当为了改进未来的**可维护性或可靠性**，或为了给未来的改进奠定更好的基础而修改软件时，出现了第四项维护活动。这项维护活动通常称为预防性维护，目前这项维护活动相对比较少。

软件维护所占百分比



■ 完善性维护 ■ 改正性维护 ■ 适应性维护 ■ 其他维护活动

从上述关于软件维护的定义不难看出，软件维护绝不仅限于纠正使用中发现的错误，事实上在全部维护活动中一半以上是完善性维护。
应该注意，**上述4类维护活动都必须应用于整个软件配置**，维护软件文档和维护软件的可执行代码是同样重要的。

软件维护的特点

结构化维护与非结构化维护差别巨大

非结构化维护——非结构化维护是指软件维护过程中**缺乏完整文档支持**（如设计文档、接口文档等），**仅依赖程序代码**进行维护的方式。非结构化维护的起点是直接分析程序代码，而非设计文档或系统架构。由于缺乏文档，难以评估软件的结构特点、数据结构、接口特性及代码改动后果，导致维护成本高昂且质量较低，而且对程序代码所做的改动的后果也是难于估量的。**这种维护方式是没有使用良好定义的方法学开发出来的软件的必然结果。**

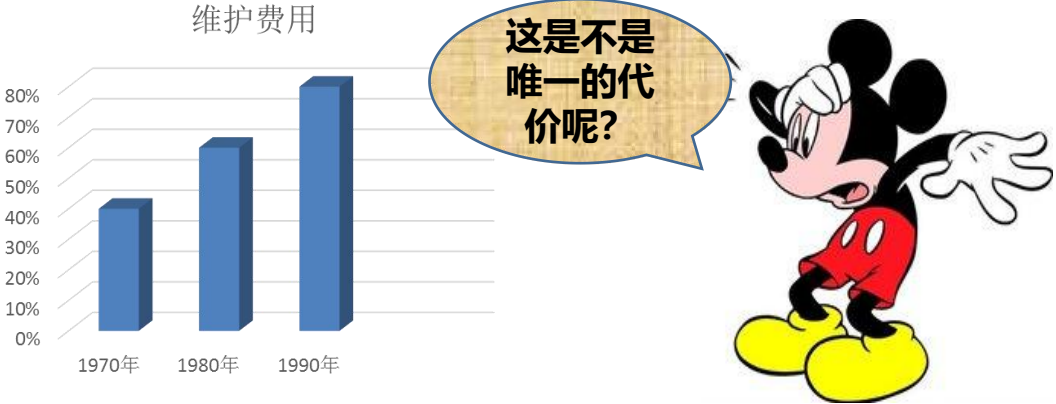
结构化维护——结构化维护是软件维护的一种方法，其核心在于通过**完整、系统的文档**支持维护过程，确保维护工作有序、高效。

- ① 要求具备完整的需求文档、设计文档和测试文档，且文档与代码高度匹配。
- ② 维护步骤包括分析软件结构、性能、接口等特性，通过文档实现代码修改的可追溯性。
- ③ 可显著降低维护成本，提高维护质量，减少错误引入

维护的代价高昂

无形的代价—— 可用资源必须供维护任务使用，以致耽误甚至丧失了开发的良机，这是软件维护的一个无形的代价。

- 当看来合理的有关改错或修改的要求不能及时满足时将引起用户不满。
- 由于维护时的改动，在软件中引入了潜伏的错误，从而降低了软件的质量。
- 当必须把软件工程师调去从事维护工作时，将在开发过程中造成混乱。

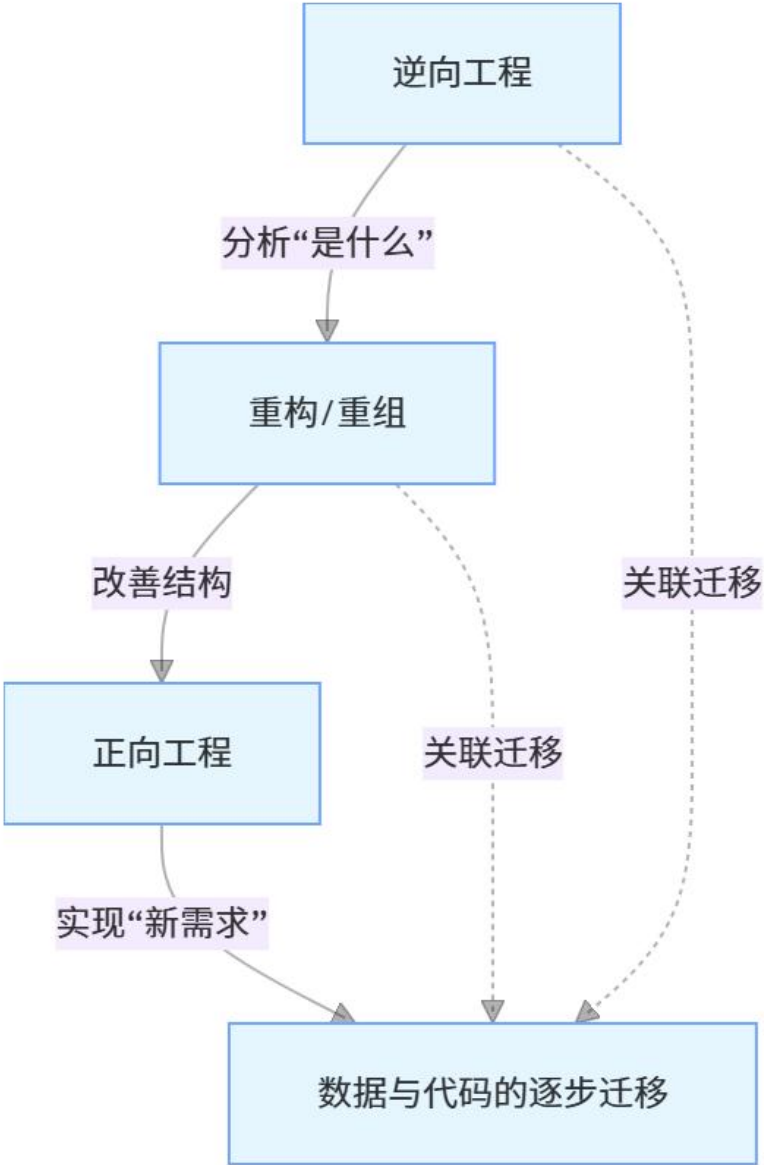


最后一个代价—— 软件维护的最后一个代价是**生产率的大幅度下降**，这种情况在维护旧程序时常常遇到。

据Gausler在1976年的报道，美国空军的飞行控制软件每条指令的开发成本是75美元，然而维护成本大约是每条指令4000美元，也就是说，生产率下降为约1/50。

报告类型	产生阶段/触发点	主要内容和目的	核心受众	
1. 维护请求/问题报告	维护入口	描述问题： 用户/测试/监控发现的问题。包含：环境、复现步骤、实际结果、期望结果、严重等级、影响范围。	维护分诊人员、维护工程师	上远在提出一项维
2. 变更请求报告	需求提出	描述变更： 对新功能、增强或重大修改的正式提议。包含：业务理由、需求描述、预期收益、粗略评估。	变更控制委员会(CCB)、产品经理、架构师	可，都需要进行同 ： 修改软件设计、
3. 维护活动/状态报告	维护过程（定期/按需）	跟踪进度： 如周报/日报。列出处理中的请求、状态（待处理、分析中、修复中、测试中）、责任人、预计完成时间。	维护经理、项目经理、相关干系人	改、单元测试和集 前的测试方案的回 和复审。
4. 故障分析/根本原因报告	重大问题解决后	深度复盘： 针对严重事故（如P0级故障）。详细分析故障时间线、根本原因、影响、临时解决措施、长期修复方案、经验教训。	技术管理层、全体相关团队（开发、运维、测试）	协调的重点不同，但 。维护事件流中最
5. 变更影响分析报告	修改实施前	风险评估： 评估拟议修改对系统其他部分、接口、性能、安全性的潜在影响。用于审批决策。	维护工程师、CCB、架构师、测试负责人	它再次检验软件配 性，并且保证事实
6. 维护工作摘要/总结报告	维护周期末（如月度、季度、版本）	全面汇总： 统计周期内处理的请求总数、按类型（纠错/适应/完善/预防）和优先级分布、平均解决时间(MTTR)、关闭率、主要成果、未决问题、趋势分析。	高层管理、维护团队、开发团队	中的要求。
7. 版本发布报告	版本发布时	发布说明： 详细列出本版本中包含的所有修复、新增功能、已知问题、升级/回滚指南、环境依赖变化。	用户、运维人员、技术支持团队	

再造工程漏斗



库存目录，

告的错误，
预期寿命，

个问题的方

文档”的方法。
没法把文档工

软件维护过程

逆向工程：是分析程序以便在比源代码更高的抽象层次上创建出程序的某种表示的过程，也就是说，逆向工程是一个恢复设计结果的过程，逆向工程工具从现存的程序代码中抽取有关数据、体系结构和处理过程的设计信息。

是一种产品设计技术再现过程，即对一项目标产品进行逆向分析及研究，从而演绎并得出该产品的处理流程、组织结构、功能特性及技术规格等设计要素，以制作出功能相近，但又不完全一样的产品。逆向工程源于商业及军事领域中的硬件分析。其主要目的是在不能轻易获得必要的生产信息的情况下，直接从成品分析，推导出产品的设计原理。

代码重构：代码重构是最常见的再工程活动。某些老程序具有比较完整、合理的体系结构，但个体模块的编码方式却是难于理解、测试和维护的。在这种情况下，可以重构可疑模块的代码。

- 用重构工具分析源代码，标注出和结构化程序设计概念相违背的部分。
- 重构有问题的代码（此项工作可自动进行）。
- 复审和测试生成的重构代码（以保证没有引入异常）并更新代码文档。

数据重构：数据重构发生在相当低的抽象层次上，它是一种全范围的再工程活动。

在大多数情况下，数据重构始于逆向工程活动，分解当前使用的数据体系结构，必要时定义数据模型，标识数据对象和属性，并从软件质量的角度复审现存的数据结构。

正向工程：也称为革新或改造，这项活动不仅从现有程序中恢复设计信息，而且使用该信息去改变或重构现有系统，以提高其整体质量。

正向工程是软件开发中将模型转换为可执行代码的关键技术环节，依托类图等UML建模工具实现系统静态结构的构建。该过程通过建立模型元素与编程语言之间的映射规则，确保设计方案向代码的准确转化

 **THANKS** 